

André Manada Correia da Silva

# TALLY

*An instrumentation library for  
fine-grained CPU scheduling of  
microprocesses*



Departamento de Ciência de Computadores  
Faculdade de Ciências da Universidade do Porto  
Julho de 2022

André Manada Correia da Silva

# TALLY

*An instrumentation library for  
fine-grained CPU scheduling of  
microprocesses*

*Relatório de Estágio Curricular*

Orientador: Bruno Loff

Departamento de Ciência de Computadores  
Faculdade de Ciências da Universidade do Porto  
Julho de 2022

## Acknowledgements

A special thank you to Professor Bruno Loff for not only dedicating part of his time to helping me throughout all the process of making this library but also never dismissing new ideas and being open to change.

# Abstract

In this project we describe a new method of preemptive context switching governed by how many CPU cycles each process executes instead of the usual execution timed context switchers.

We show how the mechanism that makes this new method possible is implemented along with the theory behind it.

We also do a complete performance analysis of the system identifying the time penalty for each one of the possible variables in the system.

# Contents

|                                                |           |
|------------------------------------------------|-----------|
| <b>Abstract</b>                                | <b>4</b>  |
| <b>List of Tables</b>                          | <b>8</b>  |
| <b>List of Figures</b>                         | <b>9</b>  |
| <b>1 Introduction</b>                          | <b>10</b> |
| 1.1 Objectives and accomplished work . . . . . | 11        |
| <b>2 An overview of the Tally library</b>      | <b>13</b> |
| 2.1 Vocabulary used . . . . .                  | 15        |
| <b>3 Implementation</b>                        | <b>16</b> |
| 3.1 Minithread . . . . .                       | 16        |
| 3.1.1 How to initialize a minithread . . . . . | 16        |
| 3.1.2 Adding code . . . . .                    | 17        |
| 3.1.3 Minithread states . . . . .              | 17        |
| 3.1.3.1 threadInUse . . . . .                  | 18        |
| 3.2 Context Switch Process . . . . .           | 18        |
| 3.2.1 Context switch basics . . . . .          | 18        |
| 3.2.2 Tally location . . . . .                 | 18        |

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| 3.2.3    | Context switch-out mechanism . . . . .                     | 20        |
| 3.2.4    | The context-switching function: switch out . . . . .       | 22        |
| 3.2.5    | The context-switching function: switch in . . . . .        | 23        |
| 3.2.6    | Exit state . . . . .                                       | 24        |
| 3.2.7    | Obtaining the tally . . . . .                              | 24        |
| 3.2.8    | Context switch-out branch predictor optimization . . . . . | 25        |
| 3.3      | Modules . . . . .                                          | 26        |
| <b>4</b> | <b>Performance</b>                                         | <b>28</b> |
| 4.1      | System Setup . . . . .                                     | 28        |
| 4.1.1    | Random Walk . . . . .                                      | 29        |
| 4.2      | Code size increase . . . . .                               | 29        |
| 4.3      | Multiple file performance hit . . . . .                    | 29        |
| 4.3.1    | Setup . . . . .                                            | 29        |
| 4.3.2    | Findings . . . . .                                         | 30        |
| 4.4      | Multiple minithreads performance hit . . . . .             | 30        |
| 4.4.1    | Setup . . . . .                                            | 31        |
| 4.4.2    | Findings . . . . .                                         | 31        |
| 4.5      | Branch prediction optimization . . . . .                   | 32        |
| 4.5.1    | Setup . . . . .                                            | 33        |
| 4.5.2    | Findings . . . . .                                         | 33        |
| 4.6      | Calculating the cost of a context switch . . . . .         | 34        |
| 4.6.1    | Overwork . . . . .                                         | 34        |
| 4.6.2    | Fixing overwork . . . . .                                  | 34        |
| 4.6.3    | Side effects of fixing overwork . . . . .                  | 35        |
| 4.6.4    | Findings . . . . .                                         | 36        |

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>5</b> | <b>Conclusion</b>                                          | <b>39</b> |
| 5.1      | Evaluating the library performance . . . . .               | 39        |
| 5.2      | Future work . . . . .                                      | 40        |
| 5.2.1    | A better estimator of CPU cycles cost . . . . .            | 40        |
| 5.2.2    | Overwork manager . . . . .                                 | 41        |
| 5.2.3    | Multi threading . . . . .                                  | 41        |
| 5.2.4    | Compatibility with other languages ans compilers . . . . . | 41        |
| 5.3      | Auto evaluation . . . . .                                  | 41        |
| <b>A</b> | <b>Code</b>                                                | <b>43</b> |
|          | <b>References</b>                                          | <b>48</b> |

# List of Tables

- 4.1 Cost in milliseconds of a context switch for some configuration of minithreads 38

# List of Figures

|     |                                                                                                                                                                                                                                                     |    |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.1 | Performance hit from using different amount of files to accomplish 1e9 work from different minithreads configurations . . . . .                                                                                                                     | 30 |
| 4.2 | Performance hit from using multiple minithreads to perform some amount of work with the number of context switches being constant and 100 cycles per execution, log log scale . . . . .                                                             | 31 |
| 4.3 | Percentage of time compared from using multiple minithreads to 1 minithread when performing some amount of work with the number of context switches being constant and 100 cycles per execution . . . .                                             | 32 |
| 4.4 | Performance hit from using multiple metacycles to perform a constant amount of work( $2^{30}$ ) for 1, 128 and 8192 minithreads and difference between using a normal context switch and a branch predictor optimized context switch(bpo) . . . . . | 33 |
| 4.5 | Amount of extra work done by each configuration of minithreads when trying to run a constant amount of work ( $2^{30}$ ) over certain metacycles .                                                                                                  | 35 |
| 4.6 | Time to execute a constant amount of work ( $2^{30}$ ) for different amounts do metacycles corrected for overwork . . . . .                                                                                                                         | 36 |
| 4.7 | Sum of runs skipped for each configuration of minithreads for low numbers of cycles per metacycles for $2^{30}$ of work . . . . .                                                                                                                   | 37 |

# Chapter 1

## Introduction

When one usually talks about CPU scheduling, one refers to the task of dividing CPU time among relatively complex computational processes, so that each might be run for a few microseconds at a time, and where each process needs to be relatively isolated by hardware controls such as memory mapping, and where we are in a situation where race conditions need to be carefully taken into account, e.g. by the implementation of locking mechanisms.

But consider a scenario where one would like to allocate computing time to tens of thousands of different tiny computational processes, where process isolation and race conditions are not an issue (e.g. they are dealt with at the programming language level, or they don't need to be dealt with at all), and where each process maintains only very small contextual information (a dozen registers, plus a very small stack). The usual multiprocess and multithreading mechanisms that are made available via operating systems have a large overhead, as they need to satisfy various requirements (isolation, memory safety, signal handling, *etc*) that this scenario does not.

Suppose further that we wish to have very fine-grained control of how long each microprocess must run. For instance, we would like to be able to tell one process to run for approximately 100 CPU instructions per second, another process should run for approximately 1000 instructions per second, and so on. This number can change for each microprocess as time goes by.

Tally it is a library that tries to solve this problem. It allows preemptive context switches governed by CPU cycles instead of by timers.<sup>1</sup>

---

<sup>1</sup>A context switch is called *preemptive* if it is decided by the thread manager that the thread should suspend execution. A *non-preemptive* context switch happens when the thread itself decides

Tally allow us to take any piece of C code instrument the code at compile time, in such a way that we can then run the function for a determined amount of CPU cycles. The instrumented code automatically suspend execution once it has exhausted the number of cycles allocated to it. We may then resume execution at a later date while allocating more CPU cycles to it. This can be done for multiple functions or even multiple instances of the same function, each one preserving their execution context (e.g. the registers, stack, heap, *etc*).

The library consists of two parts. There is a compiler plugin that instruments code at compile time, in such a way that allows for this kind of scheduling. And there is a thread manager that can compile (using the compiler plugin), load and run the instrumented code.

## 1.1 Objectives and accomplished work

Before the project started Professor Bruno already had a proof of concept for the plugin module where code could be instrumented and a separate instance where normal code could behave has instrumented code with the help of some macros on the context switcher.

There were two objectives proposed for this project:

**Objective 1:** Develop and improve the proof-of-concept instrumentation plugin

The main changes done to the instrumentation plugging were to stream line and optimize the inserted code reducing its footprint to half. Some additional features were also added like a branch predictor optimized instrumentation to try to gain some performance and a new proof-of-concept way to store the cycles needed to run but later on was removed do to complexity reasons.

**Objective 2:** Develop the context switcher to the point that it smoothly works with the instrumented code.

One of the first changes done to the context switcher was adding support for floating point operations to be saved during the context switch of a instrumented function. Support for dynamically loading a function was also added. It supports both loading a already instrumented file or receiving as input a .c file, instrumenting it and then loading it to then run. The possibility to give calling arguments to the function

---

to suspend execution.

was also an improvement. Some bugs were also fixed relating to the context switch mechanism. The last this added was support for modules, an attempt to create libraries that could be access by all running functions but that respect and use their private stacks while also being instrumented so its use could be accounted on the cycle count of each function.

Refactoring and documentation was also added to both parts of the library.

## Chapter 2

# An overview of the Tally library

Any program at some time before executing in the CPU needs to be converted to a list of assembly instructions. Despite all the pipe-lining done by modern CPUs, let us assume that one assembly instruction roughly equals one CPU cycle.

The purpose of our library is to have multiple programs, which we will call *minithreads*, executing simultaneously, with the overview of a thread manager. Each minithread should execute its instructions on its own context (including its own registers, stack, symbol table, *etc*), and the thread manager should have the ability of running a minithread for a certain chosen number of CPU instructions.

When talking about standard multi-threading the CPU has specific hardware implemented to signal when a process has executed for some amount of time, but there is no hardware solution to signal when a program as executed some amount of instructions. Because of this we have to turn to software solutions.

Here is one possible approach. Modern compilers, such as GCC, allow us to insert custom code inside the generated assembly code at compile-time. In our custom code, we can then define a variable that represents the number of instructions that a program can still execute before it needs to pause. This variable is called the *tally variable*. Then, after every instruction generated by the compiler, we can then insert custom code that decrements the tally, and checks if the tally has reached 0. If that has happened, the custom code then return the execution to the tread manager, effectively pausing the program. The code that does this will be called *context switch mechanism*.

This idea works but, if implemented naively, it becomes rather expensive computationally, since every minithread instruction would require some non negligible number

of instructions for the tally code, thus multiplying the overall number of instructions by a potentially large constant.

We may then rely on the following observation: if there is a contiguous sequence of instructions without jumps, then one may insert the tally code only at the end of the sequence. Fortunately for us, there is a specific compiler stage which allows us to do precisely this. In this stage, the code being compiled is organized into a flow control graph, in a format called GIMPLE. Each node in the graph is called a “basic block” [Gimble basic block], or `basic_block`. A `basic_block` represents a list of instructions that are executed sequentially, with a single jump at the end.

The key point is this: within each `basic_block` there are only sequential instructions without any jumps in the middle, and so we know that if we start executing a basic block it will always execute until the very end.

So, instead of running the context switch mechanism at every instruction, we can run it only at the end of each basic block. Then, instead of decrementing our variable by 1 instruction we decrement it by the number of instructions in the basic block. This simple trick allows us to significantly decrease the computational cost of our verification, while still guaranteeing that we can account for every executed instruction.

This whole process — the insertion of the tally code at the end of each `basic_block` — can be implemented by a GCC compiler plugin. Such a process, where code is modified at compile time to give it new features, is called *instrumentation*, and the code produced by such a process is called *instrumented code*.

To run a instrumented function on our thread manager we first need to load it by creating a `minithread` structure, this is done by calling `minithread_init` with a series of arguments that represent the instrumented function and its restrictions like the number of cycles it should execute once its called to run and its allowed stack size.

The thread manager itself is stateless so it does not need initialization.

Once the `minithread` is created we can run it by calling `minithread_run_cycles` with the `minithread` as a parameter. The `minithread` function will run for the number of cycles specified on its structure. The number of cycles a `minithread` can execute per run is dynamic, it can be changed when the `minithread` is paused (not currently executing).

## 2.1 Vocabulary used

Any code compiled with our plugin its called an **instrumented code**. If a function is instrumented, we will call it an *instrumented function*

The term **minithread** will be used informally to denote one of the several programs that is being concurrently run, but it will also be used in the formal sense, to refer to the data structure representing an instrumented function together with its execution context (registers, stack, *etc*).

The actual process of pausing a program execution so the thread manager regains execution control is the same used for operating systems, its called a **context switch**.

A **cycle** is the execution of one instruction on the CPU.

The **Context switch mechanism** is our new method of context switching minithreads based on how many cycles they have executed.

**Running the minithread** is the act of resuming the instrumented function of a minithread allowing it to execute for a certain amount of CPU cycles.

# Chapter 3

## Implementation

### 3.1 Minithread

Minithread is the data structure used to represent our instrumented functions and all the extra data needed to hold their context when not executing. Minithread is the main structure used to interact with our thread manager to run each instrumented function.

#### 3.1.1 How to initialize a minithread

Initializing a minithread is done by calling the `minithread_init` function. To this function we need to provide a wrapper structure that contains information about the function that the minithread is going to represent (file name where the function is located, function name and a flag that signals whether the function is already in instrumented binary or is still in a `.c` file needing to be compiled and instrumented). Further more we need to indicate the size of the memory(stack + modules) allow for the function to use and the number of cycles that the function can run when executing it, optionally we can also provide a pointer to its initial arguments and an array of module arguments structures (section 3.3) available to the function and its size.

### 3.1.2 Adding code

If the file is a .c extension then it should be added to the folder ./code. If it is a instrumented binary it should be added to ./dl.

### 3.1.3 Minithread states

Every minithread has a state variable that allow us to know in what phase of execution it is at.

- **MINITHREAD\_NEW** When a minithread is first initialized it will be in this state. Its stack will have size 0.
- **MINITHREAD\_RUNNING** Before doing a context switch to a minithread's function the thread manager will change the minithread state to running.
- **MINITHREAD\_FORCE\_YIELD** After a context switch back to the thread manager this will be the normal state for the minithread.
- **MINITHREAD\_INTERRUPTED** This state is caused by the instrumented function calling the macro **MINITHREAD\_INTERRUPT**, by using this macro the thread does an immediate context switch to the thread manager and the next time the minithread is resumed it will only have the remaining amount of cycles left when the macro was called instead of resetting its tally to its normal value.
- **MINITHREAD\_VOLUNTARY\_YIELD** This state is similar to **MINITHREAD\_STOPPED**, but caused by calling **MINITHREAD\_YIELD** and the next time the minithread is resumed its tally resets, in the same way as in **MINITHREAD\_PAUSED**.
- **MINITHREAD\_ERROR** This state is caused by the instrumented function calling **MINITHREAD\_ERROR**, this causes an immediate context switch back to the thread manager and this minithread will be unable to resume execution, this macro can be used then detecting a catastrophic failure.
- **MINITHREAD\_RETURNED** This state is set by the thread manager when detecting that a minithread has executed a return from its function and has finished executing.

### 3.1.3.1 `threadInUse`

This is a `_thread_local` variable that always refers to the minithread running. If no minithread is running the variable is null. This variable gives a self reference for macros and module functions that need information about the minithread itself.

## 3.2 Context Switch Process

In this section we will describe and explain the mechanisms that allow us to control for how many cycles the code runs and the procedure to stop and resume the minithread's execution while preserving their context.

The actual code used to do the context switch can be found in Section A.1.

### 3.2.1 Context switch basics

A context switch is the process of storing the execution context of an running program A (registers, stack, heap) effectively pausing it, and to restore the execution context of another program B, effectively beginning or resuming it.

### 3.2.2 Tally location

Remember: *tally variable* is the name of the variable responsible for representing the number of cycles left to run by the current minithread. *tally code* is the name of the code that updates the tally variable and conditionally passes execution onto the thread manager if the tally variable becomes non-positive.

In our current implementation the tally variable is stored in the AMD64 CPU register r15. In order for this to work, our instrumented code must be compiled with the GCC flag `-ffixed-r15` that tells the compiler to not use this register, thus ensuring that only the tally code can interact with the tally variable.

Initially the plan was to have two different ways to store the tally variable, the r15 register and to use a common C style variable for compilers that do not allow to reserve registers (e.g. LLVM). This would serve as a proof of concept of one less thing to modify if the plugin was to be rewritten for other compilers.

This second method raises some complications. As the library is built for multithreading and the instrumented code at compile time should be agnostic to where is going to run, the tally variable would need to be a `_thread_local` variable. This is a problem when trying to refer to the variable in our assembly code because different threads have the same named variable with different addresses so we cannot hardcode an address for our variable. The solution found was to declare in the instrumented code a variable named `counter` as `extern`, the instrumentation mechanism would then refer to `counter` using the symbols table address with `@GOT-PCREL(%rip)` ([Stackoverflow <symbol>@GOTPCREL(%rip) explained]). As `counter` is `extern` the symbol is not present at compile time leaving the reference in the code to be dealt with when linking.

When loading the code to be run we can fake the address of our `counter` variable by linking the code with the flag `-T` and giving `counter` the address of the `_thread_local` tally for the thread that is going to run the instrumented code. Now we have instrumented code that refers to the right tally variable, the same instrumented code can run in different threads by linking different tally addresses. This is not a problem when using `r15` as its address is hard-coded and each thread as one.

This second method was cut from the library at the last second before performance testing as we felt that there were too many drawbacks that influenced other parts of the library.

We did this because of the following issues.

**Code density:** The number of instructions needed to check and decrease the tally variable jumped from 2 instructions when using `r15` to 10 instructions when using the C style variable because of the trickery needed in assembly to get the counter address. This not only would increase binary sizes but also time spent by the CPU running the tally code.

**Compilation:** As referred above using a C style variable when trying to build a multithreading library adds a step of linking the instrumented file to the tally variable of a thread. One side effect of this is once the instrumented code is loaded it can only run on the thread referred in the linking process.

**Modules:** The concept of modules is described in (3.3), but in this context we can think of them as just instrumented libraries. As referred in compilation because of the C style variable the instrumented code gets locked into a thread, the same happens to the modules code. As a result, each thread will need a copy of all modules used. The reality is much worse because of the symbols table used for the function names

in the modules, i.e. different threads with two versions of the module but the same function names. The solution found was to bundle each module used in the binary of the instrumented code, this way the function names would be resolved internally with the big drawback of needing as many copies of the modules as different instrument files.

In the end, faced with all these drawbacks, it was best to leave the C style tally variable behind, improving the library by:

**Simpler loading:** Now the library does not need to touch the instrumented code before its loaded to run, all the compilation and linking can be done externally and the same file can be used in multiple threads in multiple machines without any modification.

**No more module versions:** Now we can include all the module code with the rest of the library when compiling reducing the size and complexity of the system.

**Multi-threading and load balancing:** With the instrumented code being agnostic to which thread is running opens the possibility of simpler multithreading and even using a load balancer to equal the amount of work each thread needs to do by shuffling the minithreads between threads.

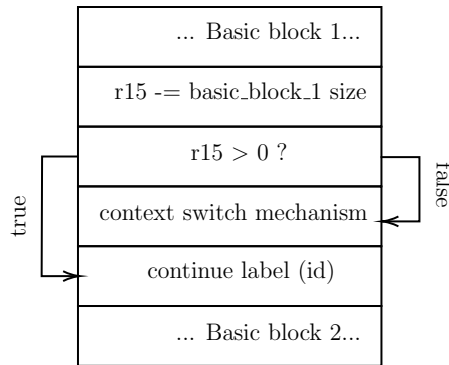
### 3.2.3 Context switch-out mechanism

Let's think about the mechanism that we must implement to be able to call a context switch out of the minithread code.

First, we need to check if its time to do the context switch, we do this by checking if our tally is less or equal to 0 meaning that we have ran out of cycles of execution.

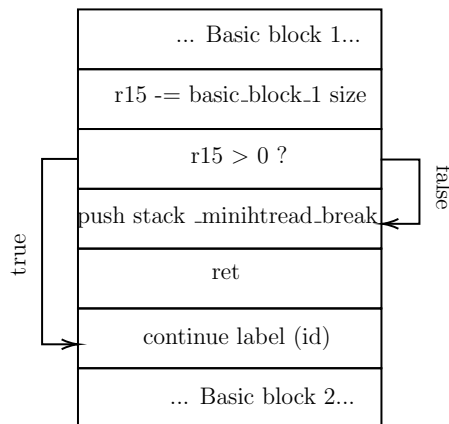
|                           |
|---------------------------|
| ... Basic block 1...      |
| r15 -= basic_block_1 size |
| ... Basic block 2...      |

If it is not time then we should skip the rest of the tally code instructions and continue the original code. We do so by doing a conditional jump to a “continue label”. **For our context switch mechanism to work, every continue label must have a unique id.**

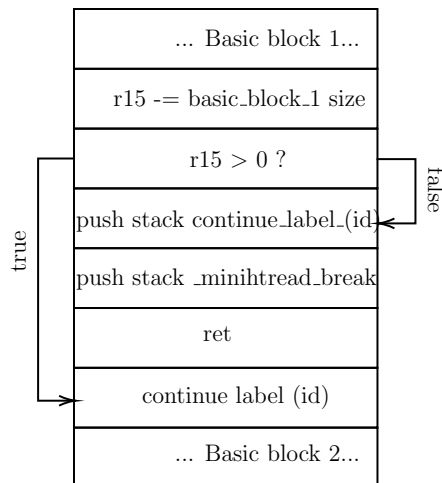


If the tally variable is negative or zero, then it's time to do the context switch, and we call the switch function. We do so by way of a “goto” instruction to a specific point in the thread manager's code, with the `_minithread_break` label. This is similar to calling a function, but it preserves the stack and every register other than the instruction pointer (IP).

This “goto” instruction can be done by pushing the `_minithread_label` onto to the stack and then executing a return instruction. A return instruction removes the top of the stack and puts that value on the instruction pointer. The context switching function of the thread manager, which contains the `_minithread_break` label, will be explained below.



In the above description of the context switch mechanism, we are still missing the address from where the context switch was activated. This is necessary so that we can switch back. This problem is then solved with the help of the continue label, since it gives us the exact point where we want to resume the execution of the minithread. Putting that label on the stack *before* the `_minithread_break` label causes the continue label to sit at the top of the stack, once the return instruction is executed. The continue label is precisely the address of the instruction that we need to go to if we wish to continue the execution of the minithread.



### 3.2.4 The context-switching function: switch out

`_minithread_break` is the label that represents the point in the thread manager's context switch function, where the execution context of the minithread is saved, and the execution context of the thread manager is restored. Please look at Section A.2, where we list the C code for the context-switch function. Here we are interested in the second half of this C code, beginning with the `_minithread_break` label, which is goto'ed whenever a minithread pauses execution.

At the `_minithread_break` point (line 20), the context-switch function starts by calling the macro `PUSH_REGISTERS`, which pushes all of the minithread's registers to the top of the minithread's stack. This needs to be the first step, since any code written in C will use the various available registers, and so any such code would alter the execution context of the minithread. The minithread's current stack pointer is saved immediately after (line 25), using extended assembly, to a thread-local variable `.thread_sp`, which will eventually be stored in the minithread's data structure. We store the stack pointer on a variable instead of also putting it on the minithread stack to easily access it when checking the minihthead return state, (see Section 3.2.6).

Now that the environment of the minithread is stored we can change the stack pointer to point to the stack of the thread manager (line 26), and restore the thread manager's context with the `POP_REGISTERS` (A.2) macro.

Why didn't we save the IP that was on the stack? When restoring the context of the minithread we need to pop all registers from the stack to their correct place and this will leave us with the correct continue label address on top. Then we may resume the

minithread simply by calling a return instruction, and this will jump to the correct place in the instrumented code.

### 3.2.5 The context-switching function: switch in

By giving control to the instrumented function and expecting control to come back we need to save the environment of the thread manager, which will later be restored when the minithread pauses execution.

We can save our environment by using a similar procedure as in the context switch-in, by only inverting which stack pointer and registers are saved and restored. Also, we need to consider the case when the minithread is starting execution for the first time, since then its stack will not have any prior saved state registers to pop. Naturally, the thread manager's registers need to be saved on the thread manager's stack, and the minithread registers will be in the minithread's stack.

If we are executing the minithread for the first time, we simply call the appropriate function pointer (line 10), which is stored in the minithread. We may do this simply using C code, as it even allows us to pass arguments to this initial call, while remaining compatible with calling conventions, *etc.*

If, instead, we are resuming the execution of the minithread we first need to use the POP\_REGISTER (A.2) macro to restore its environment (line 14). As per the context-switch protocol, this will leave the appropriate continue label on top of the stack, and thus, to resume the minithread execution, we only need to execute an assembly return instruction (line 16).

By placing the context switch-out code (`_minithread_break` label) after the context switch-in code, we also resolve the case where the instrumented function exits by its own means like by executing a return type instruction.

In the current implementation of our switch-in function we use a `_thread_local` variable to pass values from before to after switching stacks. We need to do this for a minithread reference after switching stacks is needed. This can be done because at least on *Linux* and using *Gcc* as `_thread_local` variables are not stored on the current context stack. As an alternative for when this assumptions are not true we also provide a different implementation where instead of `_thread_local` variables register variables are used to keep the reference to the minithread.

### 3.2.6 Exit state

By looking at the minithread stack state we can see if the control flow arrived at `_minithread_break` because of a context switch-out (i.e. a forced or voluntary yield, or an interrupt) or by a return instruction (meaning, the minithread function itself returned).

If the function itself has returned, this means that it consumed the bottom of the minithread's stack by calling `ret` on the label placed that was placed there when the manager called the function for the first time (line 10 of the code A.1). So at that point the minithread's stack is empty. Then, the `PUSH_REGISTERS` instruction (line 23) will add the registers on top of the minithread's stack. This takes up 30 words on the stack. So, if the function returned by itself, the stack will have size exactly 30. If, instead, the minithread was paused (by a yield or interrupt), then after the switch-out the minithread will have a stack strictly larger than 30 (because it contains at least 30 registers plus the return address for the function call).

So if the stack has size 30, the exit state is `MINITHREAD_RETURNED`. Otherwise, it is one of the paused states: `MINITHREAD_FORCED_YIELD`, `MINITHREAD_VOLUNTARY_YIELD`, or `MINITHREAD_INTERRUPT`. If the thread did a voluntary yield or an interrupt, this operation already changed the minithread's state accordingly. If, however, the minithread did a forced yield, then the state of the minithread after the context switch-out is `MINITHREAD_RUNNING`, and it is then changed to `MINITHREAD_FORCED_YIELD`.

### 3.2.7 Obtaining the tally

As mention above in Chapter 2 we take advantage of GCC GIMPLE code structure of basic blocks to minimize the number of instructions in the context switching mechanism, while at the same time ensuring we count every single instruction that is executed.

We now know how the context switch-in and switch-out are implemented. But the following important detail was left hidden under the rug: how does one estimate the number of instructions in each `basic_block`?

The issue is that, at the GCC compiler GIMPLE stage, each “instruction” is not a basic assembly instruction, but rather it is a GIMPLE instruction. These GIMPLE instructions are not exactly assembly instructions: they are potentially more

complicated. e.i. A complex variable assignment (`var x = 2 * f(3,4) + a`) is only one GIMPLE instruction but has many sub operations where each requires multiple assembly instructions.

As a result, a single GIMPLE instruction can correspond to several assembly instructions in the final assembly code. So we have no hope of learning the exact number of assembly instructions which will appear in a given basic\_block at this stage of compilation. The best that we can get is a rough estimate.

By iterating through every basic block and analysing not only the number of instruction contained in each block but also the number of operators each one has we can better estimate the CPU cycles cost of every block.

### 3.2.8 Context switch-out branch predictor optimization

Modern CPUs use a technique of branch prediction to try to speed code execution [Wikipedia branch predictor] by trying to guess which path of an if-else or conditional jump instruction the code is going to follow.

The [Intel® 64 and IA-32 Architectures Optimization Reference Manual] manual references, in its Section 3.4.1.2, what branch is predicted when encountering a conditional jump:

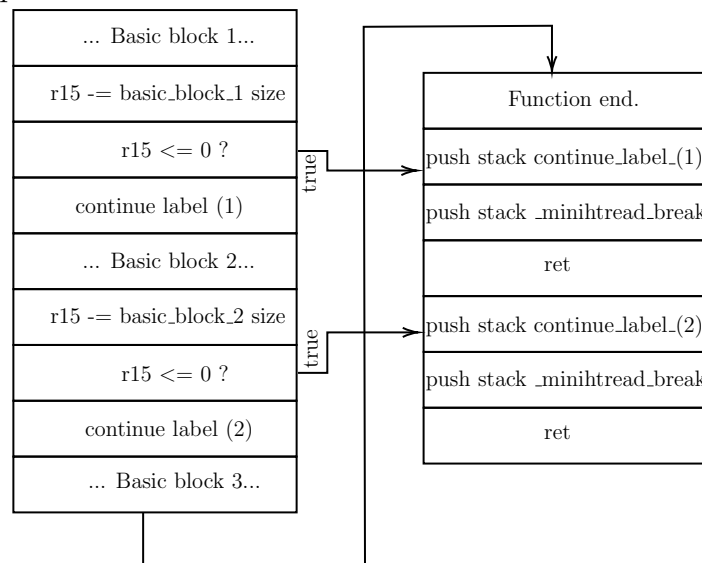
- Predict forward conditional branches to be NOT taken.
- Predict backward conditional branches to be taken.

Using the mechanism described above, and with the knowledge of the preferred branch of the branch predictor, we find that the branch predictor, if following the above rule of not taking forward branches, will always predict in favor of the context-switching code, instead of the rest of the minithread's code.

We can change this as follows. We first split the mechanism into two parts: (1) the decrement of the tally variable and the conditional jump, and (2) the switch-out code. We keep part 1 in the same place as before. And we place all the different part 2s (the switch-out code part, there is one such part for every basic\_block) at the end. (Because of the way the compiler plugin works, all the switch-out codes corresponding to each function are placed together at the end of the function.) This is illustrated in the figure below.

Then, if the tally is zero or negative, we jump to the corresponding switch-out code (part 2). This is now a forward jump, hence the branch predictor will predict against the jump, and instead predict in favor of the minithread's own code.

Using this knowledge we can try to optimize our context switch mechanism so the branch predictor always takes the path to execute more code instead of evaluating the mechanism. This is helpful as we predict that more times than none we are continuing execution of the minithread's code, instead of executing a switch. This causes branch prediction and speculative execution to favor the minithread's own code.



### 3.3 Modules

Ideally, we would now do an instrumented compilation of the entire C standard. However, this would be a massive effort, and is beyond the scope of this project. More generally, the program using the minithread library might wish to provide its own functionality to the minithreads it runs. So we implemented a module system, which is designed to give minithreads access to a controlled and limited set of shared libraries (which may in turn be instrumented).

One of the first justifications to implement modules was to allow dynamic memory allocation inside the minithreads' code. If we use the normal system calls of malloc and free this memory is not counted for the total amount of memory used by each minithread. So we decided to implement a heap module where it uses some of the the stack space of the minithread for dynamic memory allocation. We adapted a opensource project called SHMALL [Simple Heap Memory ALlocator].

The only other module we implemented was a graph module that allowed us to share a graph with all minithreads with a simple API.

# Chapter 4

## Performance

After the library was built, we needed to characterize its performance against all variables in the system.

These are some definitions that are used all during the analysis:

**Metacycle** → calling `minithread_run_cycles` on every minithread in the program once

**Number\_of\_total\_cycles / Work** → amount of cycles ran during the program execution from all minithreads.

**Number\_of\_cycles** → cycles ran by a minithread during a call from `minithread_run_cycles`.

### 4.1 System Setup

All benchmarks were performed on a Intel Xeon x5650 2.66ghz and 12MB of cache, running Ubuntu 20.04.4 LTS x86\_64 with 16gb of ram.

Every minithread was created with  $1 \ll 10$  (1024) bytes of total memory and ran “./code/random\_walk.c” on a 1000 node graph (graph.txt) so it would stay in cache the whole time.

Every data point was generated with a median of 100 samples, the data points as well as the scripts used to gather the data will be present on the GitHub page of this library (<https://github.com/ILBSPT/Tally>).

### 4.1.1 Random Walk

Random walk was the algorithm chosen to test the performance of our library.

It's a simple infinite loop that starts on a node of the graph and every iteration chooses a random neighbor to that node to jump to. On the next iteration it will choose a neighbor from that new node to jump to and so on.

## 4.2 Code size increase

Our plugin on average using small functions increased the binary file size of the functions by 100 to 200%.

## 4.3 Multiple file performance hit

Our library relies on `dlopen` and dynamic libraries of instrumented code, so we predicted that this was where we might encounter the first bottleneck in the system, as the number of files increases there might be cache misses on code fetch when switching contexts. We tested if there was a difference between multiple minithreads using the same library file versus each minithread using its own library file (with the same function code).

### 4.3.1 Setup

We started with a constant number of minithreads (1e4), doing a constant amount of work, 100 cycles per minithreads for 1000 metacycles, then for each run we varied the number of different files opened and used, but always running the same function.

If `dlopen` is called on the same file twice, it is going to return the same pointer, so we cannot have two copies of the same library file open. Hence, for this experiment to work we needed to compile the file to a shared library and then create 10000 copies of it with different filenames.

We needed to stop the data collection at 10k copies as (our version of) `dlopen` gives segfault when trying to open more than circa 13k files.

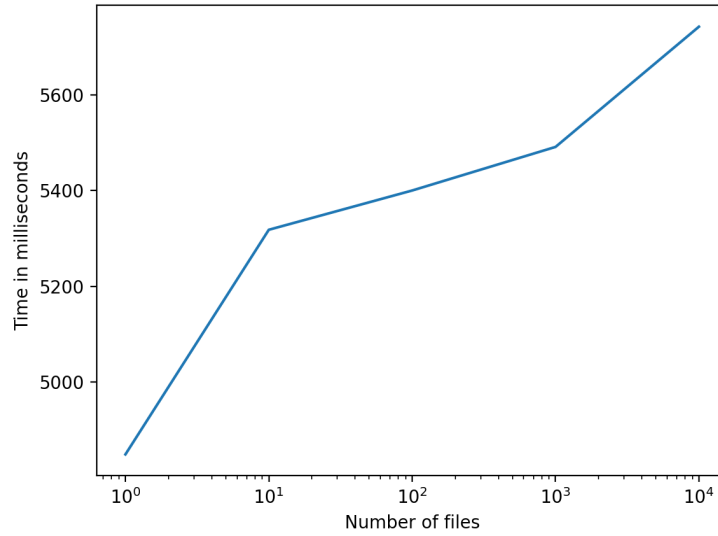


Figure 4.1: Performance hit from using different amount of files to accomplish 1e9 work from different minithreads configurations

### 4.3.2 Findings

We found just an 18% increase in time taken when using 4 orders of magnitude more files, figure 4.1. Because of this finding all the data presented bellow will be gathered using the same file for every minithread as we felt that the 18% increase is not enough to justify the added complexity of testing with multiple files.

## 4.4 Multiple minithreads performance hit

The next bottleneck that we needed to investigate was the penalty of using multiple minithreads as was our hypothesis that as the number of threads increases, we start to take cache miss penalties as we are unable to keep every minithread stack in cache.

We also tested if shuffling the order of execution of the minithread would have any kind of performance impact. This is to test if malloc allocates memory in a way beneficial to sequential reading.

### 4.4.1 Setup

To test this we ran 4 different setups with different amounts of total work:  $1e7$ ,  $\dots$ ,  $1e10$ . Each minithread did 100 cycles per metacycle and because our only intention was to study the impact of the cache misses we used the formula:

$$\text{number-of-metacycles} = \frac{\text{work}}{\text{number-of-minithreads} \times \text{number-of-cycles}} \quad (4.1)$$

to calculate the number of metacycles we needed to run and **keep the total number of context switches constant**, this way the only variance is the number of minithreads used.

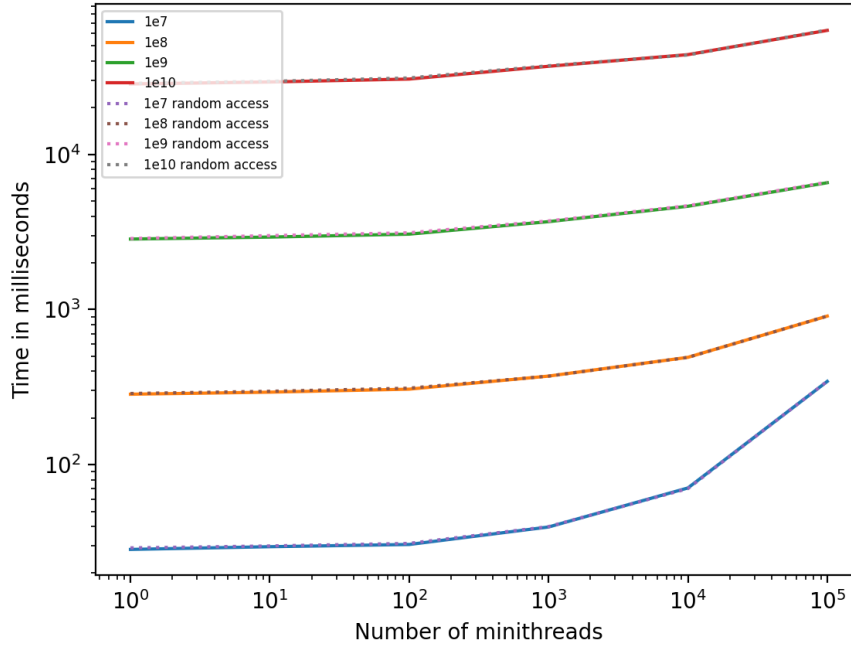


Figure 4.2: Performance hit from using multiple minithreads to perform some amount of work with the number of context switches being constant and 100 cycles per execution, log log scale

### 4.4.2 Findings

We can see (figures 4.2, 4.3) than the amount of time taken is proportional to the amount of work to do with 1 minithread, we can also see that when using multiple

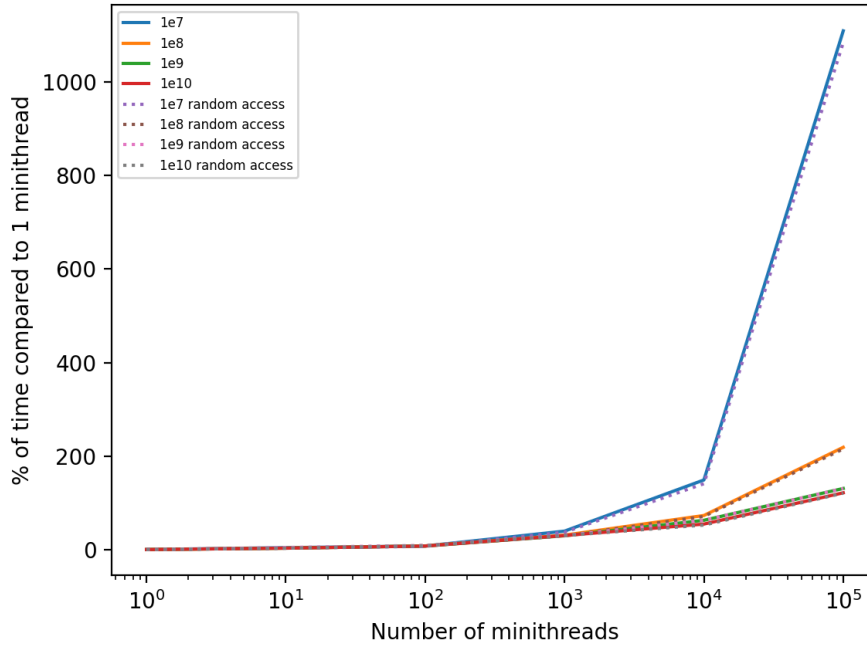


Figure 4.3: Percentage of time compared from using multiple minithreads to 1 minithread when performing some amount of work with the number of context switches being constant and 100 cycles per execution

minithreads there is a greater performance hit went doing less work, running the minithreads on a shuffled order does not yield different results this leads us to believe that the order from which the minithreads are initiated does not impact their performance .

## 4.5 Branch prediction optimization

During the phase of development, we implemented a different order for the instructions that instrument the code so we could try to work with the branch predictor and sway the prediction to the branch that we thought has the right one most of the time. The whole description and theory behind the implementation is described in section 3.2.8.

### 4.5.1 Setup

To test if it works, we ran head-to-head comparisons of the same number of threads doing work ( $2^{30}$  cycles) on a varying amount to metacycles, by varying the number of metacycles we are controlling the number of context switches done giving us the performance hit of the context switches in both instrumentation's. The number of cycles ran by each minithread per metacycles was given by this formula:

$$\text{number-of-cycles} = \frac{\text{work}}{\text{number-of-minithreads} \times \text{number-of-metacycles}} \quad (4.2)$$

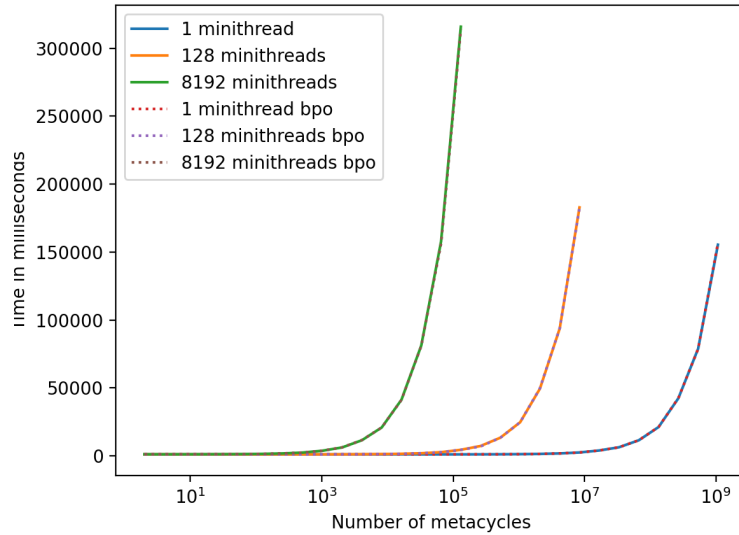


Figure 4.4: Performance hit from using multiple metacycles to perform a constant amount of work( $2^{30}$ ) for 1, 128 and 8192 minithreads and difference between using a normal context switch and a branch predictor optimized context switch(bpo)

### 4.5.2 Findings

This (figure 4.4) shows that a multi-billion-dollar company can outsmart a third-year project. There is no significant gain or loss in performance when trying to optimize for the branch predictor: it does equally well with both strategies we used.

**Attention:** the overall performance for both setups is wrong and will be discussed why bellow, but for a head to head comparison it is still valid.

## 4.6 Calculating the cost of a context switch

Calculating the cost of a context switch is important to know the practical lower limit of cycles that a minithread can run without spending more time during the switch than actually running the minithread.

But before calculating this number we need to do some corrections on our context switch graph 4.4.

### 4.6.1 Overwork

The results when running high numbers of metacycles intrigued us because of a sudden loss in performance. Upon closer inspection we suspected that our minithreads were doing overwork (more work than the total work they were supposed to do) when the number of cycles per metacycle was closing on 0. This can happen because TALLY does not have enough granularity to only do a very small number of cycles per metacycle, because as it only checks the tally after each basic\_block 3.2.7.

We can expect an overwork of:  $Cycles\_to\_run \% average\_basic\_block\_cost$  every time the run the minithread.

So if we do a large amount of metacycles with only a few cycles each, we will end up with a negative tally on most metacycles, and so the minithread will do more work than what it was supposed to do.

We can check how much extra work is being done by examining the tally of the minithread after each execution: if the tally is negative, then the minithread did more work than it should have done. If we add the absolute value of all these negative numbers, we end up with an estimate of how much overwork the minithread did.

As we suspected figure 4.5 shows that at low number of cycles per metacycles we are 6 x'ing the amount of work that was asked for, inflating the time of execution.

### 4.6.2 Fixing overwork

To stop our minithreads of doing overwork we matched each minithread to a variable that keeps track of the work done by that minithread, we can take the work asked to the system and divide it with the number of minithreads getting the work expected to

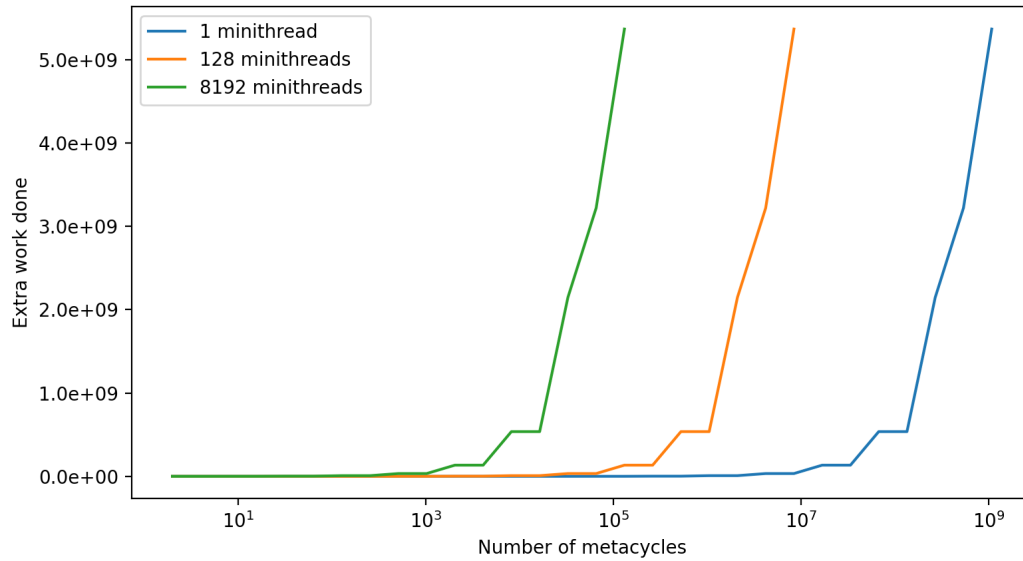


Figure 4.5: Amount of extra work done by each configuration of minithreads when trying to run a constant amount of work ( $2^{30}$ ) over certain metacycles

be done by a single minithread, when the variable of a minithread exceeds this number we stop executing it for the rest of metacycles as it has done all the work asked.

This method still leaves some overwork proportional to the number of minithreads running and the resolution on the instrumentation but it is now orders of magnitude smaller than the work asked to perform so we can ignore it.

With our overwork problem solved we ran again the same benchmark but we didn't include the results for branch optimized instrumentation as it would double the time it took to gather the data and from the last results it was evident that would not change only by removing extra work as it doesn't change the number of context switches. As predicted (figure 4.6) the extra work had impact on the performance lowering the time to 1/6th, in line with the finding on figure 4.2.

### 4.6.3 Side effects of fixing overwork

Our skip method has one flaw when trying to measure the number of cycles that a context switch takes. Because we are skipping context switches when a minithread as done its expected work 4.6.2, the actual number of context switched done is much lower.

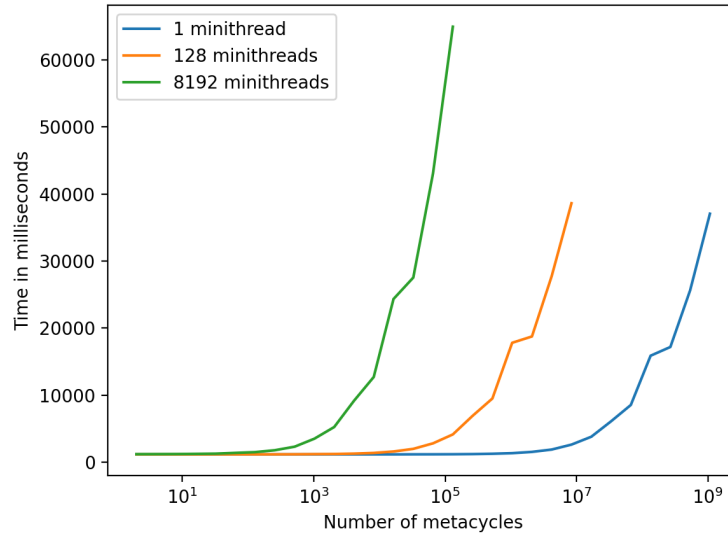


Figure 4.6: Time to execute a constant amount of work ( $2^{30}$ ) for different amounts do metacycles corrected for overwork

By analyzing our fix for all the setups we got the number of context switches skipped (figure 4.7):

This data (figure 4.7) can be also used to approximate the resolution that gcctally was capable of instrumenting. by dividing the number of context switches that was supposed to run and the real number when doing 1 cycle per metacyles we find the average size of a basic\_block for the random\_walk function used,  $\sim 6$  cycles per basic\_block, witch corresponds to doing 6 times the work when trying to do 1 cycle per metacycle without correction.

#### 4.6.4 Findings

Now with all the corrections done we can finally calculate the cost of doing a context switch.

First we did a visual estimate of the cost by instrumenting our run function and locking at the assembly code produced as adding up all the tally sub instructions. This plus the macros to save and restore registers gave us a base line of 265 cycles.

With this we can assume that our context switch is between 100 and 1000 cycles.

We also need a base line time of the function running without context switched. We

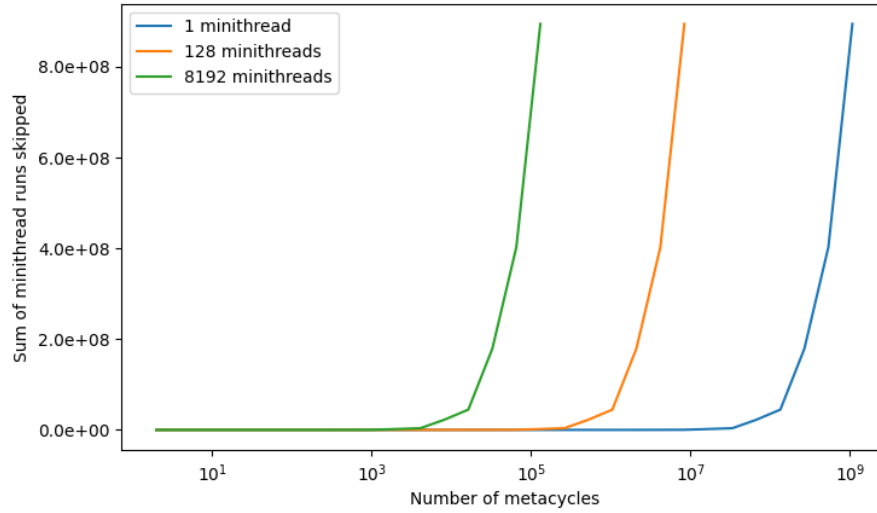


Figure 4.7: Sum of runs skipped for each configuration of minithreads for low numbers of cycles per metacycles for  $2^{30}$  of work

can get this by averaging the times on figure 4.6 for 1 minithread for the first couple of data points as the cost of context switching up to 32 times is insignificant based on our estimate for its cycles.

This gets us a time of 1165 milliseconds to perform  $2^{30}$  cycles of work in 1 and 128 minithreads and 1213 for 8192 minithreads.

Examining the data points where the number of cycles per metacycles is low enough that the context switch cost has an impact, when each minithread is performing 1024 cycles or less per metacycle, we can determine the time cost for a context switch with this formula:

$$\frac{t - 1175}{c} \quad (4.3)$$

where  $t$  is the time taken to perform the work and  $c$  the number of context switches performed.

By averaging the results for each setup of minithreads we get this result (table 4.1).

By knowing the time that it takes to perform a context switch and knowing that  $2^{30}$  cycles takes 1165 milliseconds we can determine how many cycles it takes to perform a context switch (table 4.1), we can justify the different in costs for the different

| Num. of minithreads             | 1       | 128     | 8192    |
|---------------------------------|---------|---------|---------|
| Milliseconds per cycle          | 1.08e-6 | 1.88e-6 | 1.12e-6 |
| Milliseconds per context switch | 3.85e-4 | 4.0e-4  | 4.99e-4 |
| Cycles per context switch       | 355     | 375     | 441     |

Table 4.1: Cost in milliseconds of a context switch for some configuration of minithreads

configurations by the performance hit examine by running multiple minithreads (figure 4.2).

# Chapter 5

## Conclusion

### 5.1 Evaluating the library performance

So how good is the Tally library? We think that there are two main factors to consider when evaluating it.

- **Fairness** Fairness is how precise the actual instrumentation process can represent the actual cycles executed by the each function.

Because we use the same model to estimate every instrumented function it is reasonable to assume that the cost for the same kind of instruction in one instrumented function is the same as in another instrumented function.

This cost might not be a true representation of the number of cycles used by this instructions (as discussed in 3.2.7 the instrumentation can only estimate the CPU cycle cost of each instruction) but it is a linear estimate of the value consistent throughout every instrumentation done.

- **Granularity**

This is how fine-grain can we go on the number of cycles asked to be ran by a minithread.

The granularity is given by two factors, the average cycle cost of a basic block, (this was one of the problems encountered when trying to measure the second factor of granularity, see 4.6.1 ) and the cost of each context switch.

The average cycle cost can depend on the type of code we instrument, during our testing we found a average cost of 6 cycles when running the random walk, a

simple and short function but when testing with some Competitive programming solutions (where code is highly linear) we found 18-22 cycles of average cost of basic block (this was done by visual inspection of the resulting assembly code).

We found the cost of a context switch to be 350-450 cycles (table 4.1) depending on the number of minithreads running.

All this means that except with highly linear code the major factor determining the granularity of the library is the context switch. One of the things that can help lower this cost and elevate the granularity is to not support float operations as not only this adds a lot of instructions to save these registers but also the instructions needed are a lot slower when comparing to accessing normal registers.

The maximum granularity that this library can achieve on the current state is of about 1500 cycles (counting the cost of the context switch) without beginning to be affected by issues like overwork of the performance hit of performing the context switches.

In the end we found that the current implementation of this library meets the expectations. Although the granularity is not as fine as desired, the library can support a large amount of concurrent instrumented programs with a very small overhead (section 4.4) when respecting its granularity. The cache performance of the library is also very predictable helping estimating performances without ever needing to run simulations.

## 5.2 Future work

During the execution of the project some ideas were raised but because of the time constrained were in the end left for future expansions.

### 5.2.1 A better estimator of CPU cycles cost

As we noted the GIMPLE representation is not the best to estimate the number of assembly instructions present on a linear block of code.

The next step is to build our own control-flow graph from the assembly code produced by a normal compilation. This way we have the exact amount of instructions performed.

Our new instrumentation plugin would insert code on this assembly code directly before finishing the compilation.

### 5.2.2 Overwork manager

As one of the bigger issues with our library is the overwork performed when trying to run less cycles than our granularity permits. One of the ways of fixing this when running multiple times the same minithread is to implement a system that over time does not let the minithread execute during multiple runs so it can compensate all the extra work done on previous runs.

### 5.2.3 Multi threading

One of the best ways of raising performance is to use more cores. Implementing an thread manager capable of executing multiple minithreads at the same time on different cores is one of the ultimate goals of the library. None of the implementation decisions made until now prevents this from happening.

### 5.2.4 Compatibility with other languages and compilers

Although we show this library being used in Gcc with C code, the library's goal is to be able to instrument all compiled languages. Writing a plugin works with LLVM inserting the same mechanisms on would be a great step forwards.

## 5.3 Auto evaluation

This was my first experience not only doing a bigger sized project but also writing a library for other people to use. Throughout the project I learned new technologies like using Cmake for staged compilation or debugging registers and context switches with sanitizer flags and GDB.

I also consolidated a lot of knowledge gathered during my 3 years in this course, from operating system's and CPU architecture to data analysis and software architecture and of course gaining a lot more experience writing C code.

The project was challenging but the final product was rewarding, to my knowledge this type of CPU scheduling was never been done. I also achieve a lot of the proposed goal ending up with a library capable of running tens of thousands of minithreads with a low overhead.

# Appendix A

## Code

```
1     _thread_sp = thread->sp;
2
3 run    PUSH_REGISTERS;
4
5     asm volatile("\tmovq %%rsp, %0\n" : "=r" (_my_sp) );
6     asm volatile("\tmovq %0, %%rsp\n" : : "r" (_thread_sp) );
7
8     if (thread->state == MINITHREAD_NEW) {
9         thread->state = MINITHREAD_RUNNING;
10        (thread->body)->fptr(thread->init_Args);
11    } else {
12        thread->state = MINITHREAD_RUNNING;
13
14        POP_REGISTERS;
15
16        asm volatile("\tret\n");
17    }
18
19    asm (
20        "_minithread_break:\n"
21    );
22
23    PUSH_REGISTERS;
24
25    asm volatile("\tmovq %%rsp, %0\n" : "=r" (_thread_sp) : );
26    asm volatile("\tmovq %0, %%rsp\n" : : "r" (_my_sp) );
27
28    POP_REGISTERS;
29
```

```
30 thread->sp = _thread_sp;
```

Listing A.1: Context switch excerpt

```
1  #define PUSH_REGISTERS asm (\
2    "\tpushq %r14\n"\
3    "\tpushq %r13\n"\
4    "\tpushq %r12\n"\
5    "\tpushq %r11\n"\
6    "\tpushq %r10\n"\
7    "\tpushq %r9\n"\
8    "\tpushq %r8\n"\
9    "\tpushq %rbp\n"\
10   "\tpushq %rdi\n"\
11   "\tpushq %rsi\n"\
12   "\tpushq %rbx\n"\
13   "\tpushq %rdx\n"\
14   "\tpushq %rcx\n"\
15   "\tpushq %rax\n"\
16   "\tsub $128, %rsp\n" /* 16 (8 * 2) (128 bit per float register) * 8
*/ \
17   "\tmovhps %xmm0, -128(%rsp)\n"\
18   "\tmovlps %xmm0, -120(%rsp)\n"\
19   "\tmovhps %xmm1, -112(%rsp)\n"\
20   "\tmovlps %xmm1, -104(%rsp)\n"\
21   "\tmovhps %xmm2, -96(%rsp)\n"\
22   "\tmovlps %xmm2, -88(%rsp)\n"\
23   "\tmovhps %xmm3, -80(%rsp)\n"\
24   "\tmovlps %xmm3, -72(%rsp)\n"\
25   "\tmovhps %xmm4, -64(%rsp)\n"\
26   "\tmovlps %xmm4, -56(%rsp)\n"\
27   "\tmovhps %xmm5, -48(%rsp)\n"\
28   "\tmovlps %xmm5, -40(%rsp)\n"\
29   "\tmovhps %xmm6, -32(%rsp)\n"\
30   "\tmovlps %xmm6, -24(%rsp)\n"\
31   "\tmovhps %xmm7, -16(%rsp)\n"\
32   "\tmovlps %xmm7, -8(%rsp)\n"\
33   );
34
35  #define POP_REGISTERS asm (\
36   "\tadd $128, %rsp\n" /* 16 (8 * 2) (128 bit per float register) * 8
*/ \
37   "\tmovhps %xmm0, -128(%rsp)\n"\
38   "\tmovlps %xmm0, -120(%rsp)\n"\
39   "\tmovhps %xmm1, -112(%rsp)\n"\
40   "\tmovlps %xmm1, -104(%rsp)\n"
```

```

41     "\tmovhps %xmm2, -96(%rsp)\n" \
42     "\tmovlps %xmm2, -88(%rsp)\n" \
43     "\tmovhps %xmm3, -80(%rsp)\n" \
44     "\tmovlps %xmm3, -72(%rsp)\n" \
45     "\tmovhps %xmm4, -64(%rsp)\n" \
46     "\tmovlps %xmm4, -56(%rsp)\n" \
47     "\tmovhps %xmm5, -48(%rsp)\n" \
48     "\tmovlps %xmm5, -40(%rsp)\n" \
49     "\tmovhps %xmm6, -32(%rsp)\n" \
50     "\tmovlps %xmm6, -24(%rsp)\n" \
51     "\tmovhps %xmm7, -16(%rsp)\n" \
52     "\tmovlps %xmm7, -8(%rsp)\n" \
53     "\tpopq %rax\n" \
54     "\tpopq %rcx\n" \
55     "\tpopq %rdx\n" \
56     "\tpopq %rbx\n" \
57     "\tpopq %rsi\n" \
58     "\tpopq %rdi\n" \
59     "\tpopq %rbp\n" \
60     "\tpopq %r8\n" \
61     "\tpopq %r9\n" \
62     "\tpopq %r10\n" \
63     "\tpopq %r11\n" \
64     "\tpopq %r12\n" \
65     "\tpopq %r13\n" \
66     "\tpopq %r14\n" \
67 );

```

Listing A.2: Context switch macros

```

1 struct stMinithread {
2     /// stack—pointer for the minithread
3     void **sp;
4
5     ///usefull for cheking how much memory is in use (bottom of the
6     stack)
7     void **sp_base;
8
9     /// position of memory allocated for the stack (which happens to be
10    the top of the stack)
11    void **stack;
12
13    /// last position of memory used by modules, this means that sp can
14    go from stack to stack top - 1
15    void **stack_top;
16
17    ///paramm given to the initial function
18    void* init_Args;
19
20    /// ammount of memory allocated for the stack (in terms of sizeof(
21    void*)—long words)
22    uint64_t memory_size_bytes;
23
24    /// whether the thread is currently stopped, running, paused or
25    interrupted
26    MinithreadState state;
27
28    /// the function to be run when the thread starts
29    MinithreadCode body;
30
31    ///amount of cycles needed to run, this number does not change during
32    execution of the thread
33    uint64_t cycles_to_run;
34    ///amount of cycles left when execution is stopped
35    uint64_t cycles_left;
36
37    ///array of modules all malloed, shoud be sorted by name hash
38    MinithreadModules *modules;
39
40 };

```

Listing A.3: Minithread structure

Listing A.4: decrease tally from the register r15

```

sub    basic_block size, $r15

```

Listing A.5: decrease tally from the C variable counter

```
pushq    $r15
pushq    $rdx
movq     counter@GOTPCREL(%rip), %r15
movq     (%r15), %r15
sub     basic_block size, %r15
movq     counter@GOTPCREL(%rip), %rdx
movq     %r15, (%rdx)
popq     $rdx
popq     $r15
```

# References

[Intel® 64 and IA-32 Architectures Optimization Reference Manual]

<https://www.intel.com/content/dam/doc/manual/64-ia-32-architectures-optimization-manual.pdf>

[Stackoverflow <symbol>@GOTPCREL(%rip) explained]

<https://stackoverflow.com/a/66713969>

[Gimble basic block] <https://gcc.gnu.org/onlinedocs/gccint/Basic-Blocks.html>

[Wikipedia branch predictor] [https://en.wikipedia.org/wiki/Branch\\_predictor](https://en.wikipedia.org/wiki/Branch_predictor)

[Simple Heap Memory ALlocator] [https://github.com/CCareaga/heap\\_allocator](https://github.com/CCareaga/heap_allocator)